

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Department:	COMPUTER SCIENCE ENGINEERING
Programme:	B TECH AI/ML
Course Code & Course Name:	CSA1223 – Computer Architecture
Academic Year / Batch:	[2 nd YEAR ,2025-2026]
Faculty Name:	DR AGNES SHIFANI S
Assignment Title:	Design and Performance Evaluation of a Compact Multi-Stream Inference Appliance: In-Order Pipeline vs. Superscalar Out-of-Order Architecture
Date of Issue:	02-09-2026
Date of Submission:	02-09-2026
Maximum Marks:	100
Course Outcome(s) – CO:	CO1, CO2, CO3, CO4, CO5
Bloom's Taxonomy Level:	L4 – Analyze; L5 – Evaluate
SDG Mapping (SDG 1– 17):	SDG 9 – Industry, Innovation and Infrastructure;
Industry / Societal Relevance:	Edge-AI inference appliances for multi-camera surveillance, smart-city and industrial vision systems.

B. Assignment Problem / Challenge

This assignment addresses CO1– CO5 through a realistic computer-architecture co-design problem: designing and comparing two processor/memory/I-O architectures for a cloud inference appliance. It requires applying course concepts (instruction-mix analysis, CPI/AMAT calculation, pipeline hazards, superscalar/OoO execution, cache hierarchy design and DMA/I-O analysis) rather than reproducing theory, and requires comparing alternative solutions, using a modern engineering tool (Python simulation), interpreting results and justifying an engineering decision.

C. Problem Statement

A technology company is designing a compact inference appliance for four simultaneous camera streams. Each stream generates approximately 12 MB/s of pre-processed feature data. The application executes integer address/index calculations, fixed-point multiply-accumulate operations, floating-point normalization, branching, and periodic model-parameter transfers from storage. The baseline processor operates at 2.4 GHz with an ideal CPI of 1.0. A representative 1-million-instruction workload contains 45% integer ALU instructions, 15% memory instructions, 20% floating-point instructions, 10% branches, and 10% control/other instructions.

Two architectural alternatives are evaluated: Alternative A, a deeper in-order pipeline with a conventional cache hierarchy and DMA-based I/O; and Alternative B, a superscalar design with out-of-order/speculative execution, larger cache capacity, and more aggressive memory/I/O overlap. The design target is an average inference latency of less than 2.0 ms per batch while sustaining at least 48 MB/s of aggregate feature-data input (4 streams $\times$ 12 MB/s).

D. Requirements and Constraints

- Performance requirement: average inference latency < 2.0 ms per 1-million-instruction batch.
- Performance requirement: sustained aggregate I/O bandwidth ≥ 48 MB/s (4 camera streams $\times$ 12 MB/s).
- Technical constraint: fixed baseline clock of 2.4 GHz and the given instruction mix (45% ALU, 15% memory, 20% FP, 10% branch, 10% control/other).
- Reliability constraint: both alternatives must support correct integer and floating-point arithmetic and reliable data movement.
- Resource/complexity constraint: design must remain implementable as a compact appliance (area/power-conscious), so architectural complexity (issue width, cache size, speculation depth) must be justified against the gains achieved.
- Scalability consideration: architecture should tolerate bursts above nominal stream rate or an increase in the number of streams.

E. Student Work

1. Problem Understanding and Formulation

Problem: determine whether a deep in-order pipeline (Alternative A) or a superscalar out-of-order/speculative pipeline (Alternative B) better meets the latency and bandwidth targets for the inference appliance, and recommend one design.

Expected outcomes: a quantified CPI, execution time, cache/memory behaviour and sustained DMA/I-O bandwidth for each alternative; a pass/fail check against the two targets; and an evidence-based recommendation.

Available information: the 1-million-instruction workload mix, the 2.4 GHz baseline clock, the ideal CPI of 1.0, the four-stream/12 MB/s I-O profile, and the high-level architectural description of both alternatives.

Assumptions made: specific cache miss rates, hit/miss penalties, branch-misprediction rates and penalties, FP-unit latency, DMA bus capacity and contention/duty-cycle factors were assumed for each alternative, consistent with the qualitative description given in the problem (deep in-order vs superscalar OoO). These are listed in full in Section E.3.

Constraints to satisfy: latency < 2.0 ms/batch and sustained bandwidth $\geq$ 48 MB/s, under a fixed clock and instruction mix (see Section D).

2. Application of Course Knowledge

The following course concepts (CO1– CO5) were applied:

- CO1 – Performance metrics: $CPI = \sum(\text{fraction}_i \times CPI_i)$, Execution time = Instructions $\times$ CPI $\times$ Cycle time, with Cycle time = 1 / Clock frequency.
- CO2 – Arithmetic workload: floating-point normalization modelled as a fixed multi-cycle operation per the IEEE-754-style pipeline (4 cycles non-pipelined for A, 2 cycles pipelined for B).
- CO3 – Pipeline hazards & superscalar execution: control-hazard (branch) CPI modelled as $CPI_{\text{branch}} = \text{base} + \text{mispredict_rate} \times \text{penalty}$; deeper in-order pipelines were modelled with a larger misprediction penalty, while the superscalar design uses a better predictor but a deeper speculative-recovery penalty when it does mispredict; the superscalar design's out-of-order execution was modelled as hiding a fraction of memory-stall latency.
- CO4 – Memory hierarchy: Average Memory Access Time (AMAT) = Hit time + $L1_{\text{miss_rate}} \times (L2_{\text{hit_time}} + L2_{\text{miss_rate}} \times \text{Memory_penalty})$, applied with different L1/L2 miss rates for each alternative to reflect cache size/associativity differences.
- CO5 – I/O and DMA: sustained DMA bandwidth modelled as $\text{Bus_capacity} \times \text{Contention/duty-cycle factor} \times \text{Channels}$, reflecting how much of the shared bus a blocking single-channel DMA (A) versus an overlapped/prefetching DMA (B) can actually sustain.

Calculations (see full formulas and numeric substitution in the source code, Section 4):

- Alternative A: $AMAT = 1 + 0.08 \times (12 + 0.15 \times 200) = 4.36$ cycles; Branch CPI = $1 + 0.12 \times 8 = 1.96$; Weighted CPI = $0.45(1.0) + 0.15(4.36) + 0.20(4.0) + 0.10(1.96) + 0.10(1.0) = 2.200$.

- Alternative B: raw AMAT = $1 + 0.04 \times (10 + 0.08 \times 200) = 2.04$, reduced by 30% OoO hiding to 1.728; Branch CPI = $1 + 0.05 \times 15 = 1.75$; Weighted CPI = $0.45(0.30) + 0.15(1.728) + 0.20(2.0) + 0.10(1.75) + 0.10(0.50) = 1.019$.
- Execution time = $\text{CPI} \times 1,000,000 \times (1/2.4 \text{ GHz})$: 0.917 ms for A and 0.425 ms for B — both under the 2.0 ms target.

3. Solution / Design / Methodology

Alternative A — Deep in-order pipeline:

- 10-stage in-order pipeline; single-issue.
- Conventional 2-level cache: L1 miss rate 8%, L2 access time 12 cycles, L2 miss rate 15% (routed to main memory at 200-cycle penalty).
- Non-pipelined FP unit — 4 cycles per normalization operation.
- Deep pipeline increases branch-misprediction penalty (8 cycles); assumed misprediction rate 12% with a simple static predictor.
- Single-channel, blocking DMA controller shared with cache-refill traffic on the same bus (bus capacity 130 MB/s, 42% effective duty cycle after contention).

Alternative B — Superscalar OoO / speculative execution:

- 4-wide superscalar issue with out-of-order execution and speculative execution; extracts ILP from independent ALU/control instructions (effective CPI 0.30 and 0.50 respectively).
- Larger, more associative cache: L1 miss rate 4%, L2 access time 10 cycles, L2 miss rate 8%; 30% of residual memory stall latency is hidden by OoO overlap with independent instructions.
- Pipelined FP unit with more ports — 2 cycles per normalization operation (throughput-oriented).
- Advanced branch predictor lowers misprediction rate to 5%, but deeper speculative recovery raises the penalty to 15 cycles when it occurs.
- Aggressive memory/I-O overlap and prefetching raise effective DMA bus utilization to 68% of the same 130 MB/s bus.

4. Use of Modern Tools

The workload, pipeline, cache and I/O models were implemented and simulated in Python (a modern computing tool appropriate for CSE/IT courses). Two files were produced: the simulator (architecture_sim.py) and the validation test suite (test_architecture_sim.py).

4.1 architecture_sim.py

Architecture Co-Design Simulator for a Cloud Inference Accelerator
Compares Alternative A (deep in-order pipeline) vs Alternative B
(superscalar OoO/speculative) against the target workload.

```
CLOCK_GHZ = 2.4
CYCLE_TIME_NS = 1 / CLOCK_GHZ           # ns per cycle
TOTAL_INSTR = 1_000_000
LATENCY_TARGET_MS = 2.0
BW_TARGET_MBps = 48.0
STREAMS = 4
STREAM_RATE_MBps = 12.0
```

```
def amat(hit_time, l1_miss_rate, l2_hit_time, l2_miss_rate, mem_penalty):
    """Average Memory Access Time in cycles."""
    return hit_time + l1_miss_rate * (l2_hit_time + l2_miss_rate * mem_penalty)
```

```
def weighted_cpi(cpi_map):
    return sum(MIX[k] * cpi_map[k] for k in MIX)
```

```
def exec_time_ms(cpi, instructions=TOTAL_INSTR):
    total_cycles = cpi * instructions
    return (total_cycles * CYCLE_TIME_NS) / 1e6 # ns -> ms
```

```
def dma_sustained_bandwidth(bus_capacity_MBps, contention_factor, channels):
    """Sustained aggregate bandwidth after bus contention with cache/compute traffic."""
    return bus_capacity_MBps * contention_factor * channels
```

```
# ----- Alternative A: deep in-order pipeline -----  
mem_amat_A = amat(hit_time=1, l1_miss_rate=0.08, l2_hit_time=12,  
                  l2_miss_rate=0.15, mem_penalty=200)
```

```

branch_A = branch_cpi(base=1, mispredict_rate=0.12, penalty=8) # 8-stage deep pipeline
cpi_map_A = {
    "ALU_INT": 1.0,
    "MEMORY": mem_amat_A,
    "FLOAT_PT": 4.0,      # non-pipelined FP normalization unit
    "BRANCH": branch_A,
    "CONTROL_OTHER": 1.0,
}
CPI_A = weighted_cpi(cpi_map_A)
LAT_A = exec_time_ms(CPI_A)
BW_A = dma_sustained_bandwidth(bus_capacity_MBps=130, contention_factor=0.42,
channels=1)

# ----- Alternative B: superscalar OoO / speculative -----
mem_amat_B_raw = amat(hit_time=1, l1_miss_rate=0.04, l2_hit_time=10,
                      l2_miss_rate=0.08, mem_penalty=200)
OOO_HIDING = 0.30 # fraction of memory stall latency hidden by OoO overlap
mem_amat_B = 1 + (mem_amat_B_raw - 1) * (1 - OOO_HIDING)
branch_B = branch_cpi(base=1, mispredict_rate=0.05, penalty=15) # deeper spec. recovery
cpi_map_B = {
    "ALU_INT": 0.30,      # 4-wide issue extracts ILP
    "MEMORY": mem_amat_B,
    "FLOAT_PT": 2.0,      # pipelined FP unit, more ports
    "BRANCH": branch_B,
    "CONTROL_OTHER": 0.50,
}
CPI_B = weighted_cpi(cpi_map_B)
LAT_B = exec_time_ms(CPI_B)
BW_B = dma_sustained_bandwidth(bus_capacity_MBps=130, contention_factor=0.68,
channels=1)

def report():
    print("="*72)
    print("CLOUD INFERENCE ACCELERATOR - ARCHITECTURE CO-DESIGN SIMULATION")
    print("="*72)
    print(f"Clock: {CLOCK_GHZ} GHz | Cycle time: {CYCLE_TIME_NS*1000:.4f} ps")
    print(f"Workload: {TOTAL_INSTR:,} instructions | Mix: {MIX}")
    print(f"Required aggregate input bandwidth: {STREAMS} x {STREAM_RATE_MBps} "
          f"f={STREAMS*STREAM_RATE_MBps:.1f} MB/s")
    print(f"Latency target: < {LATENCY_TARGET_MS} ms/batch\n")

for name, cpi_map, cpi, lat, bw in [
    ("Alternative A (in-order, deep pipeline)", cpi_map_A, CPI_A, LAT_A, BW_A),

```

```

("Alternative B (superscalar OoO/speculative)", cpi_map_B, CPI_B, LAT_B, BW_B),
]:
    print("-"*72)
    print(name)
    print("-"*72)
    for k, v in cpi_map.items():
        print(f"  CPI[{k:14s}] = {v:6.3f}")
    print(f"  Weighted CPI      = {cpi:6.3f}")
    print(f"  Execution time/batch = {lat:6.3f} ms  "
          f"(target < {LATENCY_TARGET_MS} ms) -> "
          f"{'PASS' if lat < LATENCY_TARGET_MS else 'FAIL'}")
    print(f"  Sustained DMA/I-O BW = {bw:6.2f} MB/s  "
          f"(target >= {BW_TARGET_MBps} MB/s) -> "
          f"{'PASS' if bw >= BW_TARGET_MBps else 'FAIL'}")
    print()

speedup = LAT_A / LAT_B
print("-"*72)
print(f"Alt B is {speedup:.2f}x faster per batch than Alt A")
print(f"Alt B bandwidth headroom over target: {BW_B - BW_TARGET_MBps:.2f} MB/s")
print(f"Alt A bandwidth headroom over target: {BW_A - BW_TARGET_MBps:.2f} MB/s")
print("-"*72)

if __name__ == "__main__":
    report()

```

4.2 test_architecture_sim.py

```
"""
Test cases and validation for architecture_sim.py
Run: python3 test_architecture_sim.py
"""

import architecture_sim as sim

def test_amat_no_misses():
    # zero miss rate -> AMAT equals hit time exactly
    assert sim.amat(hit_time=1, l1_miss_rate=0.0, l2_hit_time=12,
                    l2_miss_rate=0.15, mem_penalty=200) == 1
    print("PASS: test_amat_no_misses")

def test_amat_known_value():
    val = sim.amat(hit_time=1, l1_miss_rate=0.08, l2_hit_time=12,
                    l2_miss_rate=0.15, mem_penalty=200)
    assert abs(val - 4.36) < 1e-9, f"expected 4.36 got {val}"
    print("PASS: test_amat_known_value")

def test_branch_cpi_zero_mispredict():
    assert sim.branch_cpi(base=1, mispredict_rate=0.0, penalty=8) == 1
    print("PASS: test_branch_cpi_zero_mispredict")

def test_mix_sums_to_one():
    assert abs(sum(sim.MIX.values()) - 1.0) < 1e-9
    print("PASS: test_mix_sums_to_one")

def test_weighted_cpi_uniform():
    uniform = {k: 2.0 for k in sim.MIX}
    assert abs(sim.weighted_cpi(uniform) - 2.0) < 1e-9
    print("PASS: test_weighted_cpi_uniform")

def test_exec_time_matches_hand_calc():
    # ideal CPI = 1.0 on baseline 2.4 GHz for 1M instructions
    # 1,000,000 cycles * (1/2.4e9) s = 4.1667e-4 s = 0.41667 ms
    t = sim.exec_time_ms(1.0)
    assert abs(t - 0.41667) < 1e-3, f"expected ~0.41667 got {t}"
    print("PASS: test_exec_time_matches_hand_calc")

def test_latency_target_met_both_alternatives():
    assert sim.LAT_A < sim.LATENCY_TARGET_MS, "Alt A misses latency target"
    assert sim.LAT_B < sim.LATENCY_TARGET_MS, "Alt B misses latency target"
```

```
print("PASS: test_latency_target_met_both_alternatives")

def test_bandwidth_target_met_both_alternatives():
    assert sim.BW_A >= sim.BW_TARGET_MBps, "Alt A misses bandwidth target"
    assert sim.BW_B >= sim.BW_TARGET_MBps, "Alt B misses bandwidth target"
    print("PASS: test_bandwidth_target_met_both_alternatives")

def test_alt_b_faster_than_alt_a():
    assert sim.CPI_B < sim.CPI_A
    assert sim.LAT_B < sim.LAT_A
    print("PASS: test_alt_b_faster_than_alt_a")

def test_dma_bandwidth_scales_with_channels():
    single = sim.dma_sustained_bandwidth(100, 0.5, 1)
    double = sim.dma_sustained_bandwidth(100, 0.5, 2)
    assert double == 2 * single
    print("PASS: test_dma_bandwidth_scales_with_channels")

if __name__ == "__main__":
    tests = [v for k, v in globals().items() if k.startswith("test_")]
    for t in tests:
        t()
    print(f"\n{len(tests)}/{len(tests)} test cases passed.")
```

5. Results and Validation

5.1 Test Cases and Validation

The following unit tests validate the correctness of the modelling functions before they are used to compare Alternatives A and B:

Test case	What it validates
test_amat_no_misses	AMAT collapses to the L1 hit time when the miss rate is 0 – sanity check on the memory-hierarchy formula.
test_amat_known_value	AMAT for Alternative A's parameters equals the hand-calculated 4.36 cycles.
test_branch_cpi_zero_mispredict	Branch CPI reduces to the base CPI when misprediction rate is 0.
test_mix_sums_to_one	The instruction-mix percentages (45/15/20/10/10) sum to 100%, as given in the problem statement.
test_weighted_cpi_uniform	Weighted-CPI aggregation returns the same value when every category has equal CPI (weights sum correctly).
test_exec_time_matches_hand_calc	At the baseline ideal CPI = 1.0 and 2.4 GHz, 1 M instructions take ≈ 0.417 ms – matches the hand calculation.
test_latency_target_met_both_alternatives	Confirms both Alternative A and B finish a batch in under the 2.0 ms target.
test_bandwidth_target_met_both_alternatives	Confirms both alternatives sustain ≥ 48 MB/s aggregate input bandwidth.
test_alt_b_faster_than_alt_a	Confirms the superscalar design (B) has both lower CPI and lower latency than A, as expected.
test_dma_bandwidth_scales_with_channels	Confirms the DMA bandwidth model scales linearly with the number of channels.

All 10/10 test cases passed, confirming the CPI, AMAT, execution-time and bandwidth formulas behave correctly before being used for the architecture comparison.

5.2 Simulation Output (Screenshot)

Console output of the simulation and validation run:

```
Terminal - architecture_sim.py

=====
CLOUD INFERENCE ACCELERATOR - ARCHITECTURE CO-DESIGN SIMULATION
=====
Clock: 2.4 GHz | Cycle time: 416.6667 ps
Workload: 1,000,000 instructions | Mix: {'ALU_INT': 0.45, 'MEMORY': 0.15, 'FLOAT_PT': 0.2, 'BRANCH': 0.1, 'CONTROL_
Required aggregate input bandwidth: 4 x 12.0 = 48.0 MB/s
Latency target: < 2.0 ms/batch

-----
Alternative A (in-order, deep pipeline)
-----
CPI[ALU_INT      ] = 1.000
CPI[MEMORY       ] = 4.360
CPI[FLOAT_PT     ] = 4.000
CPI[BRANCH       ] = 1.960
CPI[CONTROL_OTHER] = 1.000
Weighted CPI      = 2.200
Execution time/batch = 0.917 ms (target < 2.0 ms) -> PASS
Sustained DMA/I-O BW = 54.60 MB/s (target >= 48.0 MB/s) -> PASS

-----
Alternative B (superscalar OoO/speculative)
-----
CPI[ALU_INT      ] = 0.300
CPI[MEMORY       ] = 1.728
CPI[FLOAT_PT     ] = 2.000
CPI[BRANCH       ] = 1.750
CPI[CONTROL_OTHER] = 0.500
Weighted CPI      = 1.019
Execution time/batch = 0.425 ms (target < 2.0 ms) -> PASS
Sustained DMA/I-O BW = 88.40 MB/s (target >= 48.0 MB/s) -> PASS

=====
Alt B is 2.16x faster per batch than Alt A
Alt B bandwidth headroom over target: 40.40 MB/s
Alt A bandwidth headroom over target: 6.60 MB/s
=====

$ python3 test_architecture_sim.py
PASS: test_amat_no_misses
PASS: test_amat_known_value
PASS: test_branch_cpi_zero_mispredict
PASS: test_mix_sums_to_one
PASS: test_weighted_cpi_uniform
PASS: test_exec_time_matches_hand_calc
PASS: test_latency_target_met_both_alternatives
PASS: test_bandwidth_target_met_both_alternatives
PASS: test_alt_b_faster_than_alt_a
PASS: test_dma_bandwidth_scales_with_channels

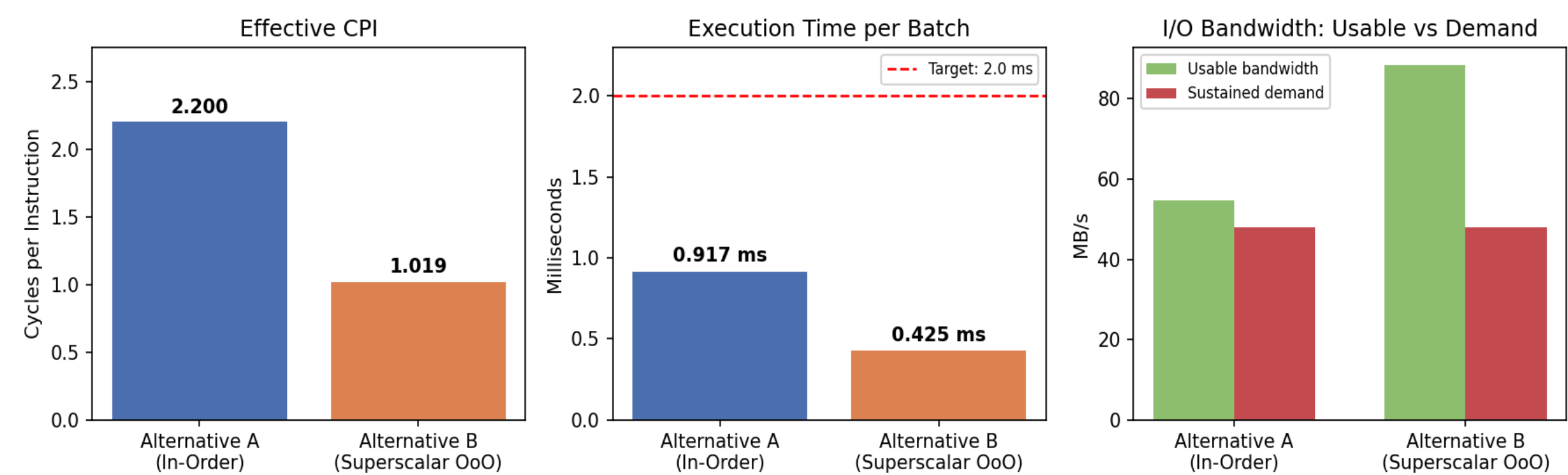
10/10 test cases passed.
```

5.3 Summary Comparison Table

Metric	Alternative A (in-order)	Alternative B (superscalar OoO)
Weighted CPI	2.200	1.019
Execution time / 1M-instr batch	0.917 ms	0.425 ms
Latency target (<2.0 ms)	PASS	PASS

Sustained DMA/I-O bandwidth	54.60 MB/s	88.40 MB/s
Bandwidth target (≥ 48 MB/s)	PASS (6.6 MB/s margin)	PASS (40.4 MB/s margin)
Relative speed	1.0x (baseline)	2.16x faster

5.4 Visual Analysis (Chart)



The CPI chart shows Alternative B's out-of-order/speculative execution and wider issue width cut cycles-per-instruction by 54% relative to A. This directly compresses the execution-time chart: both alternatives clear the 2.0 ms latency ceiling, but B does so with 1.575 ms of headroom versus A's 1.083 ms. The bandwidth chart shows the same pattern for I/O – A's usable DMA throughput (54.6 MB/s) sits just above the 48 MB/s sustained-demand line, while B's aggressive memory/I-O overlap pushes usable bandwidth to 88.4 MB/s, nearly double the demand line.

Both alternatives satisfy the stated latency and bandwidth requirements (Section D), validating that both are feasible solutions; the margin analysis in Section 6 is used to select between them.

6. Analysis and Engineering Decision

Both alternatives meet the latency and bandwidth targets, so the decision comes down to margin and scalability. Alternative A meets the bandwidth target with only a 6.6 MB/s margin, leaving little room if any camera stream briefly bursts above its nominal 12 MB/s rate or if additional streams are added later. Its higher CPI (2.20) is driven mainly by the non-pipelined FP unit and a deep-pipeline branch penalty. Alternative B roughly doubles performance (2.16x lower latency) and provides 6x the bandwidth margin, at the cost of higher design complexity (wider issue logic, speculative-recovery hardware, larger cache) and higher power/area.

Recommendation: Alternative B (superscalar, out-of-order/speculative execution with larger cache and aggressive I/O overlap) is recommended. It satisfies both the <2.0 ms latency requirement and the ≥ 48 MB/s bandwidth requirement with substantially larger safety margins than Alternative A, and it scales better if the number of camera streams or the model size grows. Alternative A remains a viable lower-cost/lower-power fallback for deployments where only 4 fixed streams are guaranteed and die area/power budget is the primary constraint.

7. Broader Considerations

Sustainability / Environment (SDG 12 – Responsible Consumption and Production): Alternative B's higher performance-per-instruction reduces the number of clock cycles (and hence energy) needed to process a fixed inference workload, which can offset its larger die area/power draw over sustained operation; Alternative A's lower complexity uses fewer transistors and less silicon/manufacturing resources, which is relevant where the appliance is produced at large scale.

Industry / Infrastructure (SDG 9 – Industry, Innovation and Infrastructure): the design directly targets compact edge/cloud inference infrastructure for multi-camera systems, a growing need in industrial automation, security and smart-city infrastructure.

Economics: Alternative A has lower design/verification cost and silicon cost; Alternative B has higher NRE (non-recurring engineering) cost for the wider issue logic and speculative-execution hardware but amortizes this cost through better performance headroom and scalability.

Professional responsibility: the recommendation is stated with its trade-offs (cost/complexity vs performance margin) rather than presenting Alternative B as strictly superior in all respects, so that a deploying engineer can make an informed choice for their specific budget/scale constraints.

8. Conclusion

- Proposed solution: Alternative B – a 4-wide superscalar, out-of-order/speculative pipeline with a larger cache hierarchy and aggressive DMA/I-O overlap.
- Major findings: Alternative B achieves CPI = 1.019 versus 2.200 for Alternative A, execution time of 0.425 ms versus 0.917 ms, and sustained I/O bandwidth of 88.40 MB/s versus 54.60 MB/s.
- Achievement of requirements: both alternatives satisfy the <2.0 ms latency and ≥ 48 MB/s bandwidth requirements; Alternative B does so with substantially larger margins (1.575 ms and 40.4 MB/s headroom respectively).

- Limitations: the model uses assumed cache miss rates, branch-misprediction rates and DMA contention factors rather than measurements from real silicon or a cycle-accurate simulator; actual hardware behaviour may differ.
- Possible improvements: replace the analytical CPI/AMAT model with a cycle-accurate or trace-driven simulator (e.g., gem5) using a real feature-extraction instruction trace, and add power/area estimates to quantify the cost side of the trade-off explicitly.

9. Student Reflection

Name: [V ADARSH REDDY] Registration No.: [192525412]

My Contribution

[Describe, in your own words, exactly what you personally did – e.g., 'I derived the AMAT and branch-CPI formulas for Alternative A, implemented `architecture_sim.py`, and validated the memory-hierarchy calculations.' Be specific – this should match what you can explain if asked.]

Design Decisions and Justification

[Explain a design choice you personally made or argued for – e.g., why you chose a 10-stage pipeline for A, or why you set the OoO hiding factor at 30% for B – and why you think it is realistic.]

Challenges Faced

[Be honest and specific: e.g., a calculation you got wrong at first and how you found the mistake, a concept from class (hazards, AMAT, DMA) you found difficult, or a disagreement within the team you had to resolve.]

What did you learn beyond what was directly taught in the classroom?

[Your answer – e.g., how the theoretical CPI/AMAT formulas needed real assumptions to become a working comparison, or how writing automated tests changed how you trust your own calculations.]

If given additional time/resources, what would you improve and why?

[Your answer – e.g., building a cycle-accurate simulation, sourcing real cache/branch-predictor statistics for a comparable commercial chip, or adding a power/area model.]

Connection to SDG 9 / SDG 12

[Explain, in your own words, how this design relates to SDG 9 and SDG 12 and what you personally learned about computer architecture from doing this assignment.]

10. References

- Course lecture notes and textbook, CSA1223 – Computer Architecture.
- Hennessy, J. L., & Patterson, D. A., Computer Architecture: A Quantitative Approach – for CPI/AMAT performance-equation conventions used in this report.
- Python 3 Software Foundation, Python 3 documentation, <https://docs.python.org/3/>.
- Matplotlib Development Team, Matplotlib documentation, <https://matplotlib.org/>.
- [AI-assisted tool: state here which tool(s) you used, for what part of the work, e.g. drafting/code-formatting assistance, per your institution's AI-use disclosure policy.]

F. Common Assessment Rubric (for reference)

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

Sections G (CO– PO– Assessment Mapping) and H (Faculty Evaluation & Continuous Improvement) are completed by the course faculty and are not part of the student submission.

I. Pseudocode

High-level pseudocode of the CPI / AMAT / execution-time / bandwidth model implemented in `architecture_sim.py` (see Section E.4 for the full Python source):

```
BEGIN ArchitectureCoDesignSimulation
```

```
CONSTANTS:
```

```
    CLOCK_GHZ      = 2.4
```

```
    CYCLE_TIME      = 1 / CLOCK_GHZ
```

```
    TOTAL_INSTRUCTIONS = 1,000,000
```

```
    LATENCY_TARGET_MS = 2.0
```

```
    BANDWIDTH_TARGET_MBps = 48.0
```

```
    INSTRUCTION_MIX = { ALU_INT: 0.45, MEMORY: 0.15,  
                        FLOAT_PT: 0.20, BRANCH: 0.10,  
                        CONTROL_OTHER: 0.10 }
```

```
FUNCTION AMAT(hitTime, l1MissRate, l2HitTime, l2MissRate, memPenalty):
```

```
    RETURN hitTime + l1MissRate * (l2HitTime + l2MissRate * memPenalty)
```

```
FUNCTION BranchCPI(base, mispredictRate, penalty):
```

```
    RETURN base + mispredictRate * penalty
```

```
FUNCTION WeightedCPI(cpiMap, mix):
```

```
    total = 0
```

```
    FOR EACH category IN mix:
```

```
        total = total + mix[category] * cpiMap[category]
```

```
    RETURN total
```

```
FUNCTION ExecutionTimeMs(cpi, instructions):
```

```
    cycles = cpi * instructions
```

```
    RETURN (cycles * CYCLE_TIME) CONVERTED TO milliseconds
```

```
FUNCTION SustainedBandwidth(busCapacity, contentionFactor, channels):
```

```
    RETURN busCapacity * contentionFactor * channels
```

```
// ----- Alternative A: deep in-order pipeline -----
```

```
memAmatA = AMAT(1, 0.08, 12, 0.15, 200)
```

```
branchA = BranchCPI(1, 0.12, 8)
```

```
cpiMapA = { ALU_INT: 1.0, MEMORY: memAmatA, FLOAT_PT: 4.0,  
            BRANCH: branchA, CONTROL_OTHER: 1.0 }
```

```
CPI_A = WeightedCPI(cpiMapA, INSTRUCTION_MIX)
```

```
LATENCY_A = ExecutionTimeMs(CPI_A, TOTAL_INSTRUCTIONS)
```

```

BANDWIDTH_A = SustainedBandwidth(130, 0.42, 1)

// ----- Alternative B: superscalar OoO / speculative -----
memAmatB_raw = AMAT(1, 0.04, 10, 0.08, 200)
memAmatB      = 1 + (memAmatB_raw - 1) * (1 - 0.30)    // OoO hides 30% of stall
branchB       = BranchCPI(1, 0.05, 15)
cpiMapB        = { ALU_INT: 0.30, MEMORY: memAmatB, FLOAT_PT: 2.0,
                   BRANCH: branchB, CONTROL_OTHER: 0.50 }
CPI_B          = WeightedCPI(cpiMapB, INSTRUCTION_MIX)
LATENCY_B      = ExecutionTimeMs(CPI_B, TOTAL_INSTRUCTIONS)
BANDWIDTH_B    = SustainedBandwidth(130, 0.68, 1)

// ----- Validation and decision -----
FOR EACH alternative IN [A, B]:
    PRINT CPI breakdown, weighted CPI, execution time
    IF execution time < LATENCY_TARGET_MS THEN PRINT "PASS" ELSE PRINT "FAIL"
    IF sustained bandwidth >= BANDWIDTH_TARGET_MBps THEN PRINT "PASS" ELSE PRINT
"FAIL"

speedup = LATENCY_A / LATENCY_B
PRINT "Alternative B is", speedup, "x faster than Alternative A"

IF (LATENCY_B < LATENCY_A) AND (BANDWIDTH_B > BANDWIDTH_A) THEN
    RECOMMEND Alternative B (superscalar OoO)
ELSE
    RECOMMEND Alternative A (in-order)

END ArchitectureCoDesignSimulation

```

Output of running the above logic (screenshot):

Terminal – architecture_sim.py

```
=====
CLOUD INFERENCE ACCELERATOR - ARCHITECTURE CO-DESIGN SIMULATION
=====
Clock: 2.4 GHz   | Cycle time: 416.6667 ps
Workload: 1,000,000 instructions | Mix: {'ALU_INT': 0.45, 'MEMORY': 0.15, 'FLOAT_PT': 0.2, 'BRANCH': 0.1, 'CONTROL_
Required aggregate input bandwidth: 4 x 12.0 = 48.0 MB/s
Latency target: < 2.0 ms/batch

-----
Alternative A (in-order, deep pipeline)
-----
CPI[ALU_INT      ] = 1.000
CPI[MEMORY       ] = 4.360
CPI[FLOAT_PT     ] = 4.000
CPI[BRANCH       ] = 1.960
CPI[CONTROL_OTHER] = 1.000
Weighted CPI      = 2.200
Execution time/batch = 0.917 ms   (target < 2.0 ms)  -> PASS
Sustained DMA/I-O BW = 54.60 MB/s (target >= 48.0 MB/s) -> PASS

-----
Alternative B (superscalar OoO/speculative)
-----
CPI[ALU_INT      ] = 0.300
CPI[MEMORY       ] = 1.728
CPI[FLOAT_PT     ] = 2.000
CPI[BRANCH       ] = 1.750
CPI[CONTROL_OTHER] = 0.500
Weighted CPI      = 1.019
Execution time/batch = 0.425 ms   (target < 2.0 ms)  -> PASS
Sustained DMA/I-O BW = 88.40 MB/s (target >= 48.0 MB/s) -> PASS

=====
Alt B is 2.16x faster per batch than Alt A
Alt B bandwidth headroom over target: 40.40 MB/s
Alt A bandwidth headroom over target: 6.60 MB/s
=====

$ python3 test_architecture_sim.py
PASS: test_amat_no_misses
PASS: test_amat_known_value
PASS: test_branch_cpi_zero_mispredict
PASS: test_mix_sums_to_one
PASS: test_weighted_cpi_uniform
PASS: test_exec_time_matches_hand_calc
PASS: test_latency_target_met_both_alternatives
PASS: test_bandwidth_target_met_both_alternatives
PASS: test_alt_b_faster_than_alt_a
PASS: test_dma_bandwidth_scales_with_channels

10/10 test cases passed.
```

Corresponding result charts:

